SYNAPSE

ADVANCED DEVELOPER MATCHMAKING AND HACKATHON COLLABORATION PLATFORM

A PROJECT REPORT

submitted in partial fulfillment of the requirements for the course

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Submitted by

2116240701409 - RAGHUL M

2116240701428 - RITHESH MADHAV S



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI - 602 105

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "SYNAPSE: ADVANCED DEVELOPER MATCHMAKING AND HACKATHON COLLABORATION PLATFORM" is the bonafide work of RAGHUL M (2116240701409) and RITHESH MADHAV S (2116240701428), carried out under my supervision for the course Foundations of Artificial Intelligence.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

Synapse is a full-stack web application for hackathon collaboration. It lets users discover team requests, publish member requirements, apply to a team, and browse external hackathon directories. The implementation uses Next.js 14, React, TypeScript, Tailwind CSS, NextAuth v5, Prisma, and a PostgreSQL schema. Its data model records profiles, skills, declared roles and experience, team requests represented as Hackathon records, applications, and user-specific external hackathon favorites. The deployed application is served by Vercel.

This report assesses the application as a Foundations of Artificial Intelligence project by separating the intended Constraint Satisfaction framing from the code that currently executes. The schema defines finite role and experience domains and stores skill arrays, which are suitable inputs for a future constraint-based matcher. The verified team-discovery code performs client-side conjunctive filtering over search text, event mode, experience label, and skills. No CSP search procedure, heuristic score, ranking model, machine-learning model, or automatic team recommendation engine was found in the inspected source.

The report documents the verified architecture, database relations, authentication, routes, API endpoints, external directory integrations, and test result.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
	3.1 ARCHITECTURE DIAGRAM	9
4	MODULES DESCRIPTION	16
	4.1 AUTHENTICATION AND ACCOUNT MODULE	16
	4.2 REGISTRATION VERIFICATION AND RESET	17
5	IMPLEMENTATION AND RESULTS	23
	5.1 EXPERIMENTAL SETUP	23
	5.2 RESULTS	25
6	CONCLUSION AND FUTURE WORK	30
7	REFERENCES	35

CHAPTER 1

INTRODUCTION

1.1 Background

Hackathons require people with complementary technical abilities to form a working team within a limited time. A participant may be able to build a user interface, train a model, write backend services, prepare a deployment pipeline, or present a prototype, but no single profile reliably covers every role. Informal discovery through social groups and event channels can be slow and can hide relevant candidates outside a participant's immediate network. Synapse addresses this discovery problem through a web interface that holds user profiles and publishes team requirements.

The project is implemented as a Next.js web application. The public interface exposes a landing page, a team directory, an external hackathon directory, and authentication pages. Authenticated flows support profile updates, creating a team requirement, applying to a team request, and maintaining external hackathon favorites. The system therefore focuses on the practical sequence that precedes collaboration: discovery, eligibility information, contact, and application.

1.2 Problem Statement

A potential hackathon participant needs to locate a suitable team request without reading every listing manually. A team organizer needs a way to express required skills, the intended role, experience level, event mode, location, and registration date. Conventional listing pages provide information but often leave the user to manually compare requirements against their own profile. The FOAI opportunity is to represent the comparison as constraints and use those constraints to reduce the search space.

1.3 Objectives and Scope

OBJECTIVE	VERIFIED INTERPRETATION
Publish team requirements	Authenticated form posts a Hackathon record with team, event, role, skills, mode, location, and date.
Discover opportunities	Team cards are fetched from /api/teams and filtered in the client when data is available.
Browse external events	Hackathon page requests Unstop, Devpost, and Devfolio directory APIs.
Manage identity	NextAuth supports credentials plus Google and GitHub providers.
Support future intelligent matching	Schema stores finite role and experience domains and skill arrays, but no automatic matcher is implemented.

1.4 Stakeholders and Assumptions

Primary stakeholders are hackathon participants, team organizers, event-focused student communities, and project supervisors reviewing the FOAI formulation. The system assumes that users enter accurate skills and contact links, that external directory APIs remain available, and that an organizer is able to evaluate applications after receiving them. The current user interface does not demonstrate a team-balance optimizer, automatic acceptance decision, reputation score, or conflict-resolution workflow.

1.5 Contributions and Current Limitations

- A full-stack evidence base with a relational schema, authentication routes, route handlers, and external directory aggregation.
- A transparent client-side rule filter that can narrow a list by search text, mode, experience label, and skill substring.
- A documented FOAI formulation that distinguishes stored candidate constraints from an unimplemented CSP solver.
- A clear limitation report: deployed team retrieval returned HTTP 500 during inspection, and protected interfaces could not be evaluated without credentials.

CHAPTER 2

LITERATURE REVIEW

2.1 Constraint Reasoning and Team Formation

Constraint satisfaction represents a problem through variables, finite domains, and restrictions that define acceptable combinations. Dechter describes constraint processing as a general framework for reasoning about assignments subject to constraints [3]. In a team-formation context, a candidate selection variable may have a domain of users, while hard constraints can require a role, availability, or eligibility condition. Soft constraints can express preferred skill overlap, timezone compatibility, or team diversity. This framework is useful for Synapse because its schema already expresses role and experience as finite enumerations and skills as arrays.

Team formation has also been studied as a skill-coverage problem. Lappas, Liu, and Terzi formulate the problem of selecting experts who collectively cover task skills while considering social-network structure [5]. Synapse does not model an expert graph or communication cost, so that method cannot be claimed as implemented. Its value here is conceptual: a future Synapse matcher could combine skill coverage with a small number of explicit constraints and explain why each team is proposed.

2.2 Recommendation and Multi-Criteria Selection

A recommender normally ranks alternatives by predicted utility, similarity, preferences, or a learned model. Multi-criteria decision procedures can combine several attributes, but the aggregation rule must be stated. The inspected code does not calculate a score, weight criteria, sort candidate matches, or train a model. It evaluates a Boolean conjunction over filtered records. The literature therefore informs a future extension rather than describing an existing recommendation component.

2.3 Web Architecture and Persistent Data

Next.js 14 documents the App Router as a file-system routing model that uses current React features [6]. In Synapse, pages, layouts, route handlers, and client components follow that approach. Prisma documents a PostgreSQL datasource through a schema provider and connection URL [8]. The inspected Prisma schema uses provider = "postgresql" and relates User, Hackathon, Application, Account, and ExternalHackathonFavorite records. These sources support the technology discussion; they are not evidence that every architectural box in the supplied diagram is deployed.

SOURCE	RELEVANCE TO SYNAPSE	IMPLEMENTED OR CONCEPTUAL
Dechter [3]	Variables, domains, and constraints provide the FOAI vocabulary.	Conceptual formulation; finite domains stored.
Lappas et al. [5]	Skill coverage and team selection motivate future matching.	Conceptual only; no expert graph or solver.
Next.js [6]	App Router pages, layouts, and route handlers.	Implemented in app/ and app/api/.
Prisma [8]	Schema-driven PostgreSQL data access.	Implemented in prisma/schema.prisma.
NextAuth [1]	Credentials and OAuth session handling.	Implemented in auth.ts and auth.config.ts.

2.4 Gap Addressed by the Project

The practical gap is not a lack of a general team-formation theory; it is the absence of a lightweight, accessible workflow that brings profile information, team requirements, and external event discovery into one application. Synapse supplies that workflow. The evidence also shows the gap between the product description and the present algorithm: a genuine intelligent matcher still needs candidate retrieval, explicit hard and soft constraints, a selection strategy, and an explanation mechanism.

CHAPTER 3

PROPOSED SYSTEM

3.1 System Overview

Synapse combines a public discovery interface with authenticated creator and applicant flows. The landing page links to team and hackathon exploration. The team area retrieves organizer-created Hackathon records. The hackathon area aggregates remote listings from Unstop, Devpost, and Devfolio. Authentication is handled by NextAuth and a Prisma adapter. The currently verified persistent layer is PostgreSQL, not MongoDB Atlas.

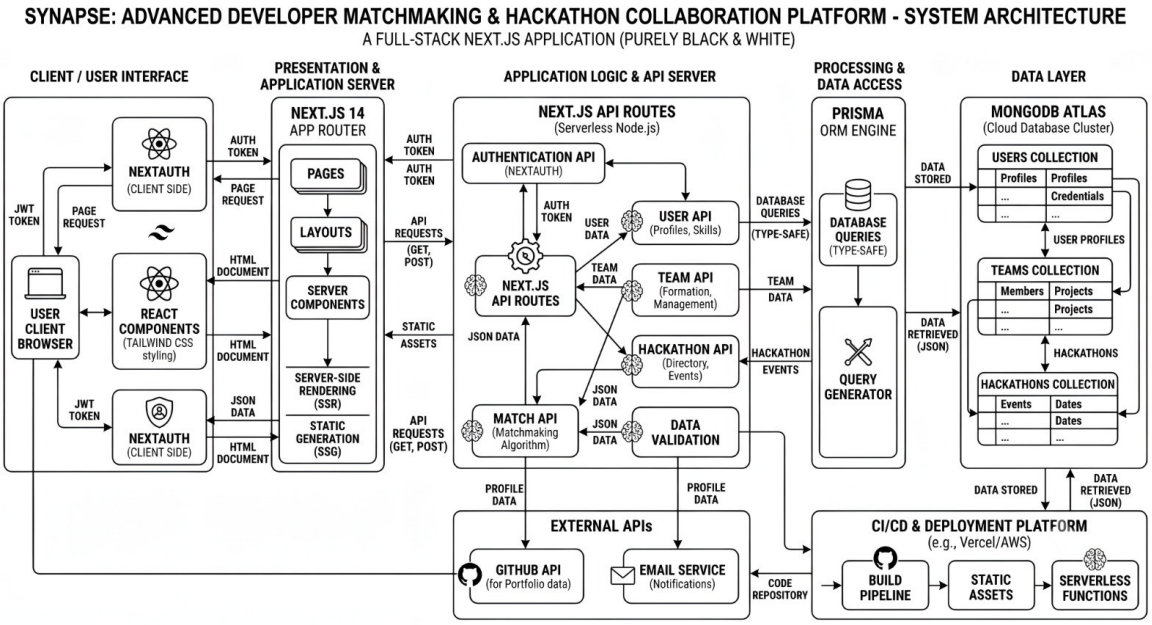


Figure 3.1 Supplied proposed system architecture. This diagram is included as supplied; it depicts intended components that are not all verified in the repository.

3.2 Architecture Comparison

SUPPLIED DIAGRAM CLAIM	SOURCE OR DEPLOYMENT EVIDENCE	ASSESSMENT
MongoDB Atlas data layer	prisma/schema.prisma specifies provider = postgresql; Docker Compose includes postgres-dev.	Not supported by current code.
Prisma ORM	lib/prisma.ts instantiates PrismaClient; route handlers call Prisma methods.	Supported.

Next.js 14 App Router	package.json uses next ^14.2.7; app/ contains pages, layouts, and route handlers.	Supported.
NextAuth	auth.ts and auth.config.ts configure NextAuth, credentials, Google, and GitHub.	Supported.
Match API and CSP algorithm	No /api/match route or solver function was found. Team filtering is in PostsTab.tsx.	Not supported as implemented.
GitHub portfolio API	Profile accepts a GitHub URL; no GitHub API call was found.	Not supported as an integration.
Email service	Nodemailer-based verification and reset helpers are present.	Supported in source; delivery not exercised.
Vercel/AWS	Response headers identify Vercel; AWS S3 code is present for optional resume upload.	Vercel verified; S3 source-only.

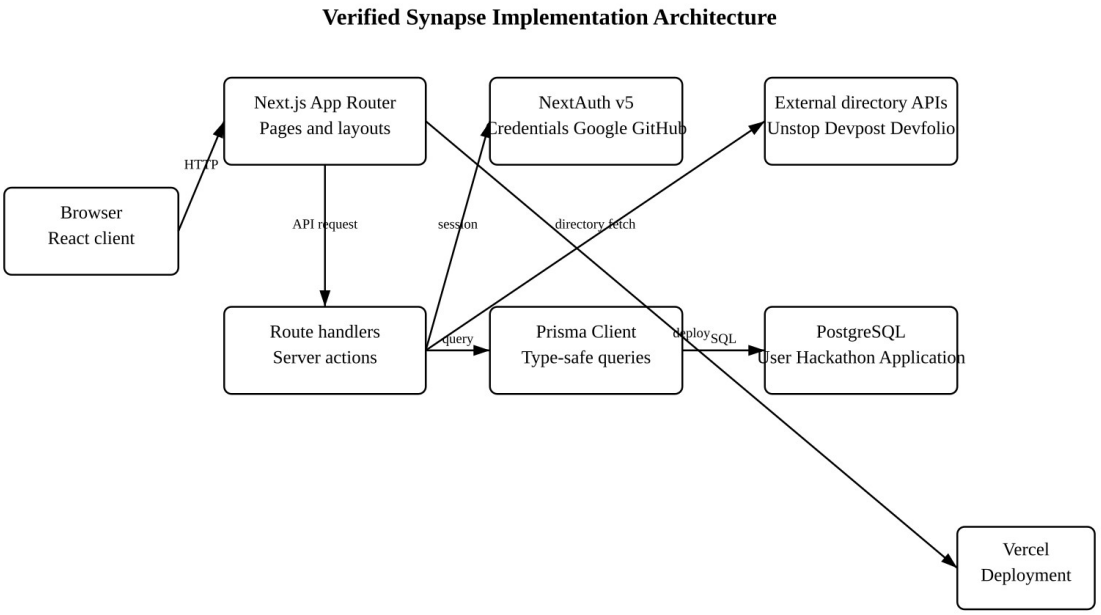


Figure 3.2 Verified implementation architecture derived from the inspected source.

3.3 User Roles and Requirements

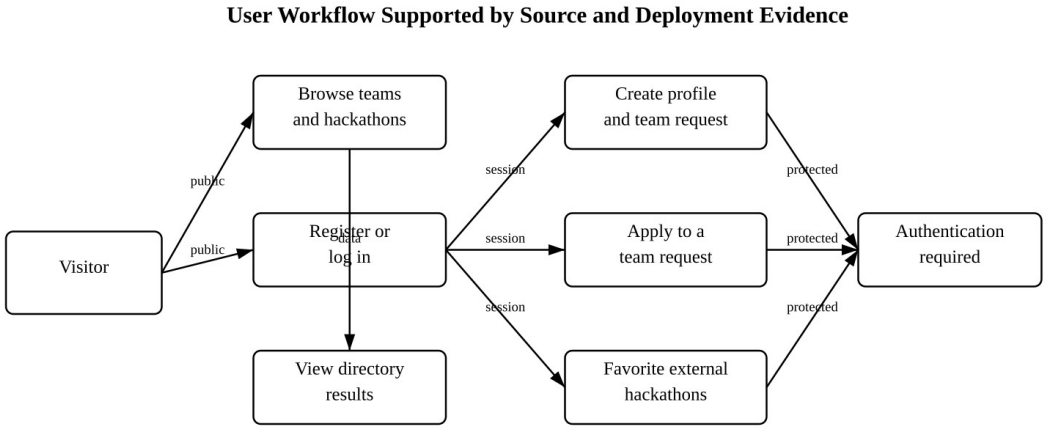


Figure 3.3 User workflow supported by source and deployment evidence.

CATEGORY	REQUIREMENT
Functional	Visitors can reach public home, teams, hackathons, login, signup, reset, and verification routes.
Functional	Authenticated users can create team requirements, update profile information, apply to team posts, and store favorites.
Non-functional	The web client is responsive through Tailwind CSS class usage and has desktop/mobile navigation components.
Non-functional	Password comparisons use bcrypt; authentication state uses JWT sessions through NextAuth.
Constraint	No session may be bypassed; protected URLs redirect to /login in the deployed inspection.
Constraint	External directory content depends on remote provider APIs and may change independently of Synapse.

3.4 FOAI Formulation and Actual Algorithm

An accurate future CSP for team matching can define candidate variables x1 ... xk, whose domains are eligible users, and hard constraints for role, event

mode, application uniqueness, and required skills. A soft objective can reward skill coverage and compatible preferences. The present code does not instantiate these variables, enumerate domains, backtrack, propagate constraints, or optimize an objective. Instead, it retrieves a list of posts and evaluates direct predicates on each post.

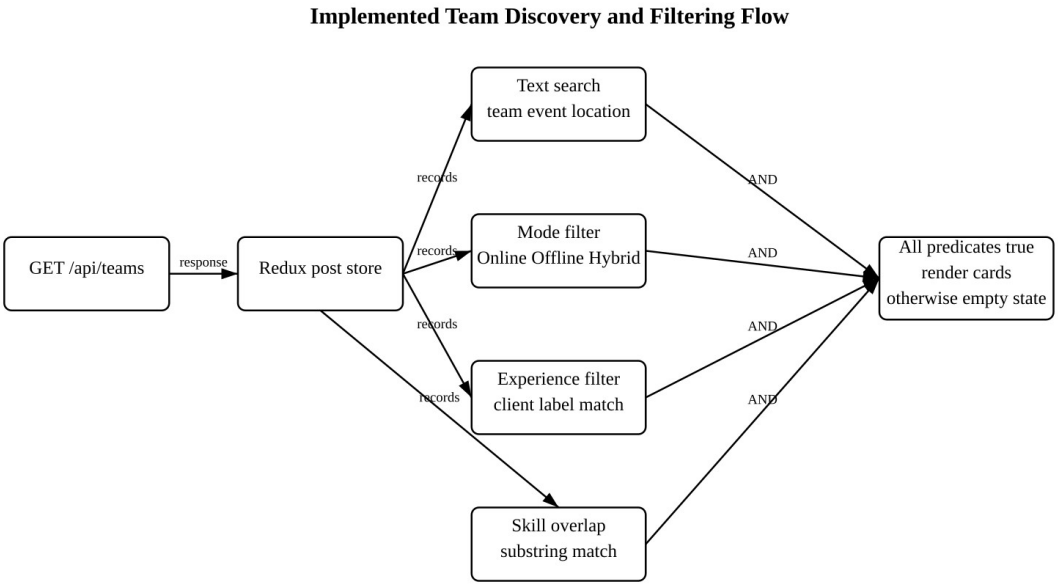


Figure 3.4 Implemented team discovery and filtering flow. The logic is a client-side Boolean filter, not a CSP solver.

```
matchesSearch    =    fields.some(value    =>    value.toLowerCase().includes(search))
matchesMode      =      selectedModes.length      ?      selectedModes.some(m      =>
post.hackathonMode.includes(m))                              :                          true
matchesExperience   =   selectedExperience.length   ?   selectedExperience.some(e   =>
post.experience.includes(e))                               :                          true
matchesSkills    =    selectedSkills.length    ?    selectedSkills.some(skill    =>
post.skills.some(s          =>          s.includes(skill)))          :          true
return   matchesSearch   &&   matchesMode   &&   matchesExperience   &&   matchesSkills
```

The output is an unranked subset of listings in their existing order. When no listing satisfies every active predicate, the interface renders an empty-state message. Matching uses case-insensitive substring inclusion. There is no tie-breaker, no explicit score, and no indication that a required role is enforced in the evaluated code; moreover, the component reads legacy role and experience

field names while the current schema defines requiredRole and requiredExperience. This inconsistency is a current limitation.

3.5 Data Model and Data Flow

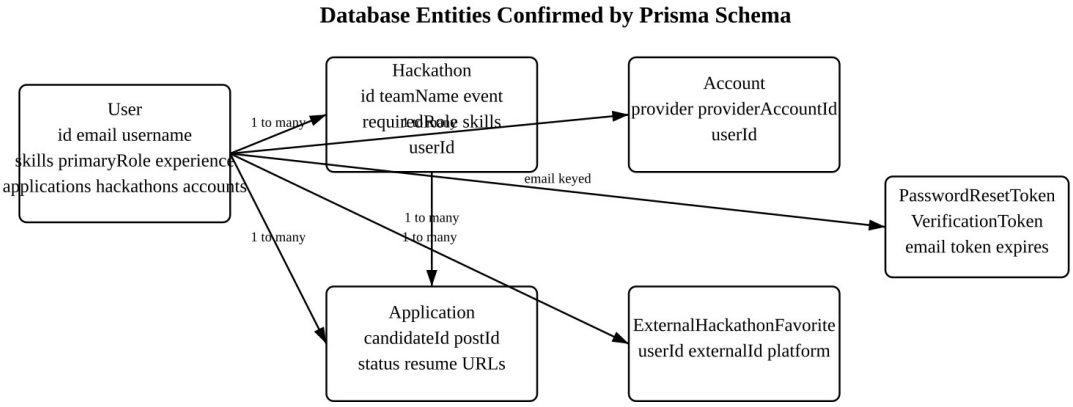


Figure 3.5 Database entity relationship diagram based on prisma/schema.prisma.

The User record is the central identity entity. It owns account links, created hackathon records, submitted applications, and saved external favorites. A Hackathon record represents an organizer-created team requirement rather than a public event directory item. It stores the team and event names, mode, location, registration date, required role and experience fields, skills, and the organizer identifier. An Application joins one candidate to one team request and is constrained to be unique for that pair. This relation prevents one candidate from applying repeatedly to the same request.

The schema uses cascading deletion from User to accounts, applications, owned hackathons, and favorites. The Application to Hackathon relation is retained so an application cannot point to a missing post. These relationships support basic integrity but do not model a separate Team entity, team membership roster, recommendation history, or interaction graph.

Consequently, the source currently supports individual requests and applications rather than a full multi-member team optimization model.

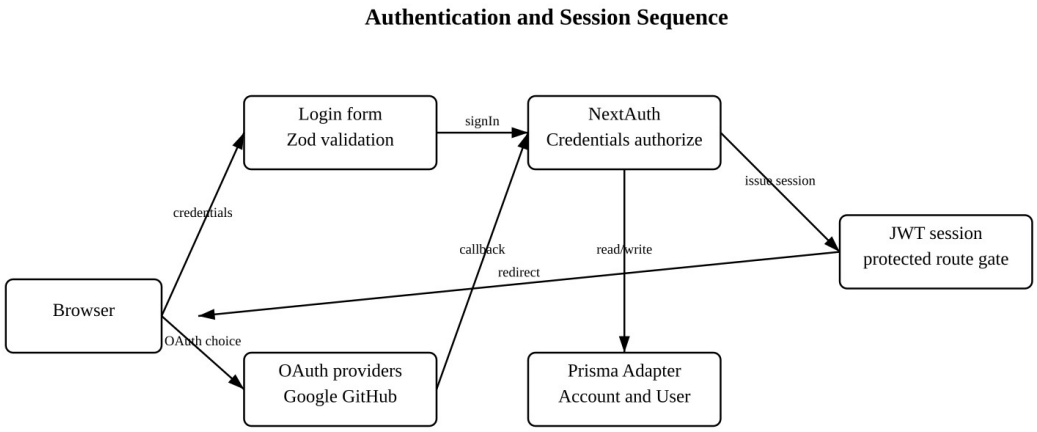


Figure 3.6 Authentication and session sequence based on auth.ts and auth.config.ts.

A credentials sign-in begins with client validation and reaches NextAuth authorization, which retrieves the account record and compares the stored bcrypt hash. On success, NextAuth stores the user identifier in a JWT session. The middleware then distinguishes public routes, authentication routes, and protected routes. The session gate is therefore the primary control for dashboard pages and creation workflows. OAuth account links are stored through the Prisma adapter when Google or GitHub authentication is configured.

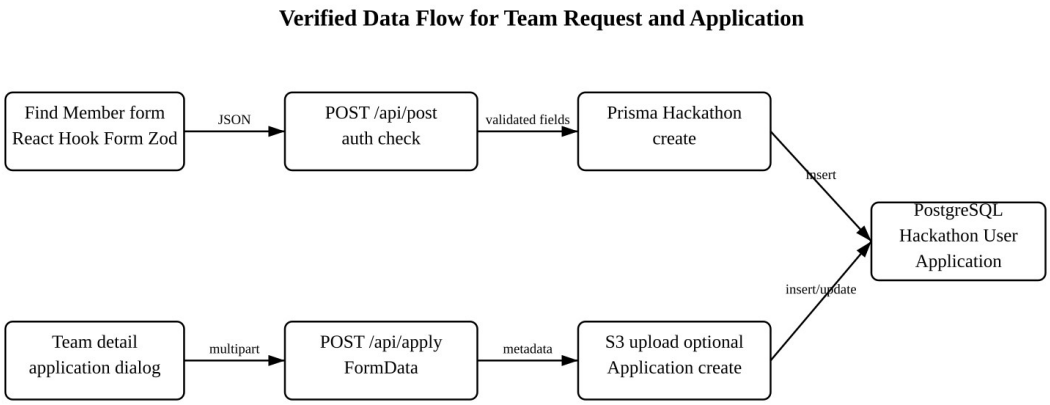


Figure 3.7 Verified team request and application data flow.

The team request form sends JSON to the post route after the user submits event and role information. The route calls auth() before it creates a Hackathon record. The application path is different because it sends multipart FormData: optional resume bytes can be uploaded to S3, while link fields and identifiers are persisted through Prisma. A production-ready version should derive every mutable user identifier from the server session and validate the request body before database operations. The report identifies this as a recommended improvement rather than a demonstrated protection.

CHAPTER 4

MODULES DESCRIPTION

4.1 Authentication and Account Module

The authentication module is defined in auth.ts and auth.config.ts. NextAuth is configured with a Prisma adapter and JWT sessions. The credentials provider validates email and password inputs with LoginSchema, looks up the user with GetUserByEmail, and compares a bcrypt hash. Google and GitHub providers are also configured through environment variables. The middleware treats auth URLs differently from public and protected routes; unauthenticated access to a protected page redirects to /login.

4.2 Registration, Verification, and Password Reset

Actions/register.ts validates fields, hashes a password with bcrypt using ten rounds, checks for duplicate email or username values, creates the user, and generates a verification token. The mail helper uses Nodemailer to prepare verification and reset messages. Actions/login.ts blocks an unverified credentials account until a verification message is sent. Actions/newPassword.ts validates a reset token, checks expiry, hashes the replacement password, and deletes the used token. These paths are present in source but their end-to-end mail delivery was not exercised because no authorized test mailbox was supplied.

4.3 Profile Module

The dashboard profile form retrieves and updates user data through /api/user. It exposes name, email, GitHub URL, LinkedIn URL, portfolio or resume link, profile image, skills, bio, and a role field. The route handler accepts an object containing user data, replaces the skills array with the submitted array, and performs prisma.user.update. The code does not show route-level authorization verifying that the submitted user.id belongs to the

current session. This should be corrected before relying on the endpoint for production authorization.

4.4 Team Request and Discovery Module

The Find Member page provides a React Hook Form containing team, hackathon, registration URL, event mode, member count, skills, experience, date, location, and description fields. It posts the data to /api/post. The route first calls auth() and rejects unauthenticated requests. It creates a Hackathon record with the current session user id. Team discovery calls GET /api/teams, stores its Hackathon array in Redux, and client-filters that array. During deployed inspection, GET /api/teams returned HTTP 500, so no current public data result can be asserted.

4.5 Application Module

A team detail page calls POST /api/id to retrieve one Hackathon record and GET /api/user to retrieve its organizer data. Its application dialog collects LinkedIn, GitHub, and resume information. POST /api/apply detects a duplicate candidateId/postId pair, optionally uploads a selected resume to an AWS S3 bucket, updates profile links, and creates an Application record. The database itself enforces the candidateId/postId uniqueness constraint. The code should additionally verify the session user at the API boundary because candidateId is accepted from form data.

4.6 External Hackathon and Favorites Module

The hackathons page requests the internal Unstop, Devpost, and Devfolio route handlers. Each handler fetches a remote provider endpoint with axios; the client normalizes selected fields and displays cards by platform. The favorites route requires auth(), then creates or deletes a user-specific ExternalHackathonFavorite using a composite unique key. Deployed requests to all three external directory handlers returned HTTP 200 during the

inspection. The favorites endpoint returned HTTP 401 without a session, which is expected.

4.7 Validation, Error Handling, and Observability

Zod schemas validate registration, login, reset, new password, and front-end team form input. Prisma operations are enclosed in try/catch blocks in the route handlers and usually return JSON error messages. Sentry configuration files and a Sentry example route are present. However, the post route imports a hackathonSchema but does not call parse or safeParse before using the request body. This leaves a gap between the intended validation contract and the executed server route.

MODULE	INPUTS	OUTPUTS	LIMITATIONS
Authentication	Credentials or OAuth callback	JWT session or error	No authorized browser session available for UI test.
Profiles	User form data	Updated User record	POST route trusts submitted user object.
Team requests	Form fields and session	Hackathon record	Live GET /api/teams returned 500.
Filtering	Search and selected filters	Unranked subset	No CSP, score, or ranking.
Applications	Links, resume, identifiers	Application record / S3 URL	API lacks visible session ownership check.
External events	Remote APIs	Normalized cards	Availability relies on third parties.

4.8 Captured Interface Evidence

Figures 4.1 to 4.13 are genuine user-supplied captures of the deployed Synapse interface. They are included as interface evidence rather than as proof of every backend operation. In particular, they show rendered controls and displayed sample content; they do not override the separately reported HTTP 500 observation for the public team-list endpoint or the source-code findings about the present filtering algorithm.

4.8.1 Landing, Theme, and Account Entry

The landing captures show the shared navigation, call-to-action buttons, theme treatment, and the collaboration message presented to a public visitor. The sign-up capture shows the three available account-entry choices: GitHub, Google, and email. The support-widget capture is identified as an embedded assistance interface and is not treated as a Synapse-authored matching module.



Figure 4.1 Dark-theme landing hero with public navigation and team-discovery calls to action.

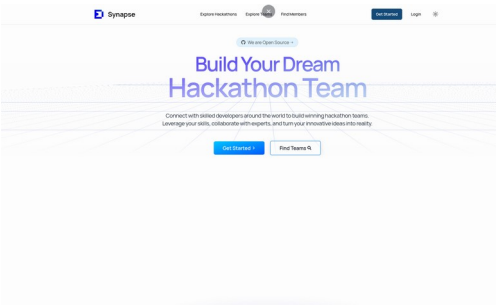


Figure 4.2 Light-theme landing hero showing the same public entry points.

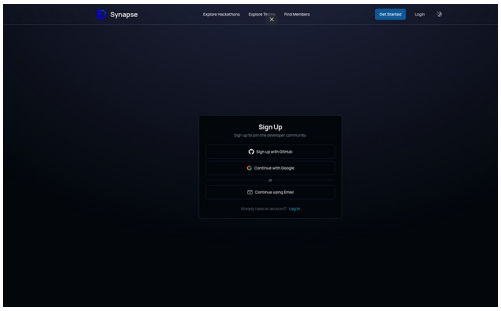


Figure 4.3 Registration screen with GitHub, Google, and email continuation choices.

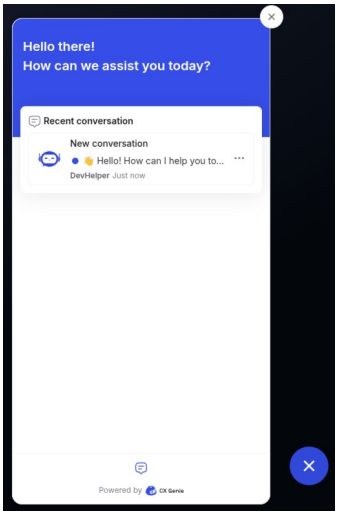


Figure 4.4 Embedded support-chat widget opened from the landing page.

4.8.2 Profile and Navigation Controls

The profile capture displays contact-link controls, a skills area, an about-me area, and an Edit Profile action. The compact navigation capture provides Dashboard, Profile, Edit Profile, Application Status, Team Requests, GitHub,

Support, and Logout controls. These screens correspond to the account and profile modules described above.

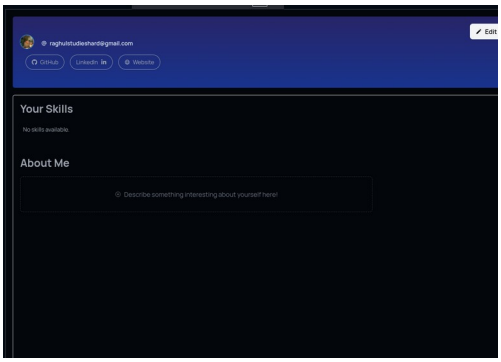


Figure 4.5 Authenticated profile view with links, skills, biography area, and Edit Profile control.

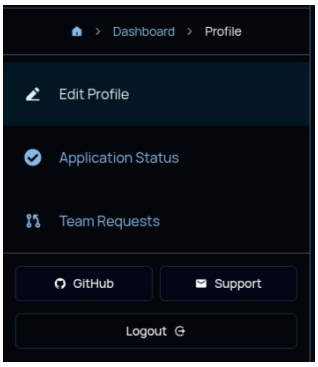


Figure 4.6 Compact dashboard navigation with profile, request, support, and logout controls.

4.8.3 Hackathon Discovery and Team Filtering

The hackathon directory capture visibly separates Unstop, Devpost, and Devfolio provider tabs and presents event cards with mode, live status, location, favorite, and visit controls. The team-discovery capture shows the actual search field and filter categories for mode, experience, and skills, alongside rendered team requirement cards. It supports the report's description of a user-driven filter interface; it does not evidence automated ranking or a CSP solver.

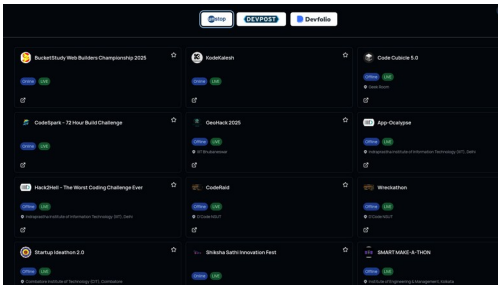


Figure 4.7 External hackathon directory showing platform tabs and event cards.

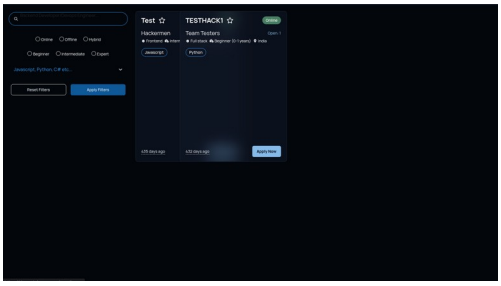


Figure 4.8 Team discovery search, mode, experience, skill filters, and requirement cards.

4.8.4 Team Request, Account Menu, and Applications

The team-request capture presents the organizer form fields for event details, member count, role, skills, experience, location, dates, and additional

details. The account menu exposes quick links to profile and dashboard views. The application-status capture renders a status table for submitted, rejected, and accepted outcomes, with a visit-post action for each displayed entry.

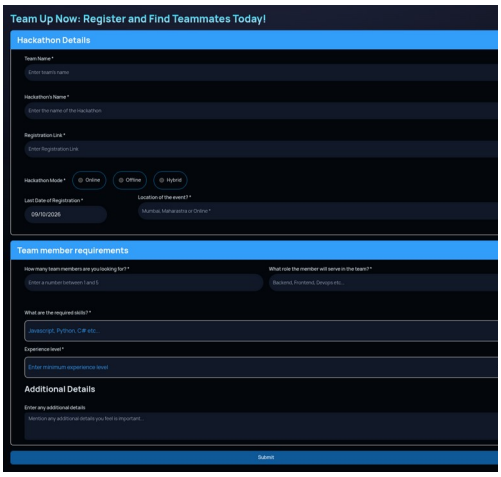


Figure 4.9 Find Member team-request form with event and team-member requirement fields.

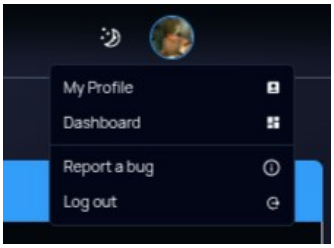


Figure 4.10 Account menu with profile, dashboard, report, and logout actions.



Figure 4.11 Application-status table with team, hackathon, visit-post, and status columns.

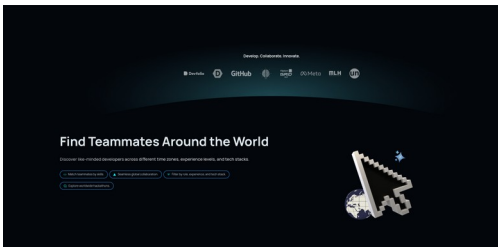


Figure 4.12 Landing collaboration section with global-discovery messages and partner marks.

4.8.5 Public Workflow and Help Content

The final landing-page capture describes the visible user journey: create a team, discover teams matching a skill set, explore upcoming hackathons, and upload a resume or connect GitHub. The lower part of the same view exposes frequently asked questions. These are public explanatory components, not an implementation of a skill-matching algorithm.

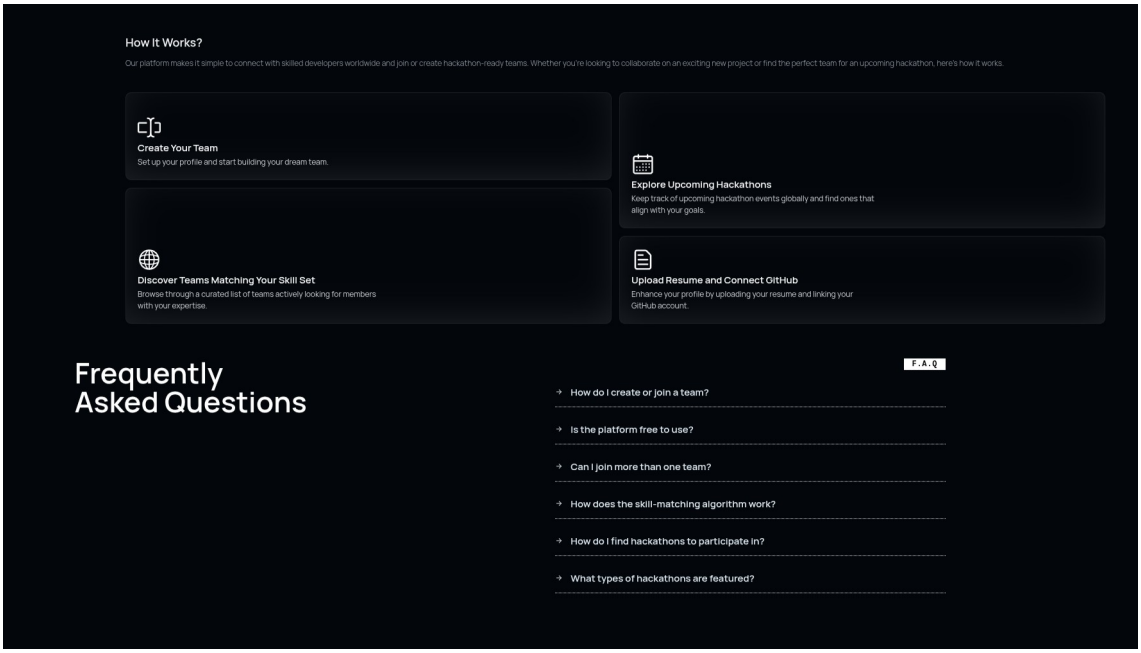


Figure 4.13 Public How It Works and frequently asked questions sections of the landing page.

The workflow cards give a visitor a concise description of the intended sequence: establish a profile, publish or discover a team need, explore an event, and provide supporting profile material. The cards are useful interface guidance because they expose the system boundaries before sign-in. They should be read as product navigation, however, rather than as execution evidence for automatic teammate selection. The displayed phrase about teams matching a skill set is therefore interpreted in this report alongside the inspected client predicates, which implement direct user-selected filtering.

The FAQ area is a second public help surface. It groups questions about creating or joining a team, platform access, multiple teams, skill matching, event discovery, and featured event types. This grouping makes the public landing page more self-explanatory, while the answer content and any dynamic interactions were not separately exercised in this report. The capture documents the interface state supplied by the project team; it does not assert a test result for every FAQ interaction.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The codebase is a TypeScript Next.js project named devmatchups in package.json. Its dependency declarations identify Next.js 14.2.7, React 18.3.1, Prisma 5.22.0, NextAuth 5.0.0-beta.18, Tailwind CSS 3.4.1, Zod 3.23.8, React Hook Form, Redux Toolkit, axios, Nodemailer, AWS SDK, and Sentry. The relational schema declares PostgreSQL, while Docker Compose provisions a postgres-dev service. The deployment response headers identify Vercel as the serving platform.

COMPONENT	OBSERVED CONFIGURATION	EVIDENCE
Frontend	Next.js 14 App Router, React, TypeScript, Tailwind CSS, shadcn-style components.	package.json; app/; components/
Authentication	NextAuth v5, credentials, Google and GitHub providers, JWT session.	auth.ts; auth.config.ts
Database	Prisma 5.22.0 with PostgreSQL datasource.	prisma/schema.prisma
State and forms	Redux Toolkit, React Hook Form, Zod.	lib/store; team and auth forms
External data	Unstop, Devpost, Devfolio APIs; AWS S3 code for resumes.	app/api/*Hackathon; apply route
Deployment	Vercel response header; Sentry build configuration.	HTTP response; next.config.mjs

5.2 Source Implementation Evidence

The public homepage composes Navbar, Hero, logos, globe, process flow, FAQ, call to action, and footer components. The application uses nested layouts to apply navigation and footer structure. Team listings and hackathon directory screens are client components because they use effect hooks, Redux,

local state, and browser interaction. Route handlers are placed under app/api/ and return NextResponse JSON values.

```
export async function GET(req: Request) {
                                    try {
                  const hackathon = await prisma.hackathon.findMany();
        return NextResponse.json({ Hackathon: hackathon }, { status: 201 });
                          } catch (error) {
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
                                                                    }
}
```

The excerpt above is the complete team-list handler in app/api/teams/route.ts. It reads all Hackathon records and returns them without pagination or an authorization requirement. Its deployed HTTP response was 500 at the time of inspection. The status code 201 for a successful GET is also semantically unusual; a successful retrieval should normally use 200. These observations are implementation findings, not assumptions about application intent.

5.3 Route Inventory and Access Results

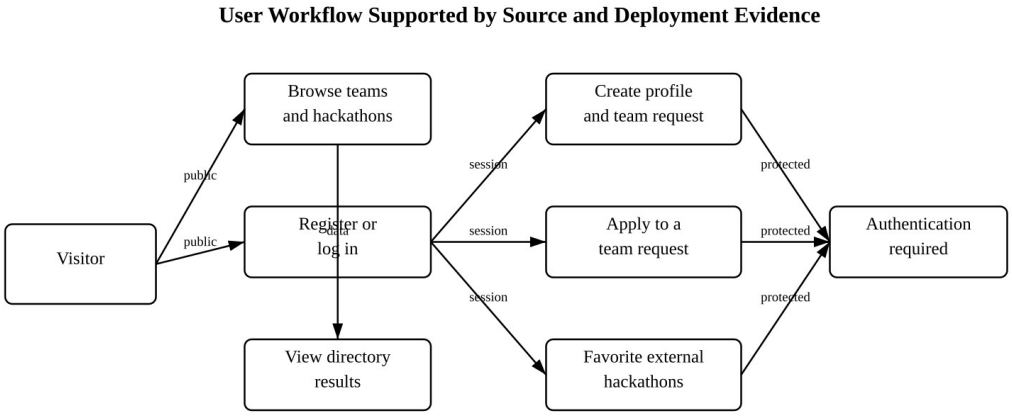


Figure 5.1 Route-access and observed-result summary. Genuine deployed-interface captures are documented separately in Chapter 4.

ROUTE	PURPOSE	HTTP	ACCESS /

		OBSERVATION	LIMITATION
/	Landing page	200; title Synapse	Public; HTML inspected.
/teams	Team discovery	200; title Synapse	Public page; backing GET /api/teams was 500.
/hackathons	External event directory	200; title Synapse	Public; each external handler returned 200.
/login	Credentials/OAuth sign in	200; title Login | Synapse	Public; form interaction not executed.
/signup	Registration	200; title Signup | Synapse	Public; no account created.
/forgot-password, /reset	Recovery pages	200	Public; no email flow exercised.
/dashboard and child routes	Profile and requests	Redirected to /login	Protected without credentials.
/findmember, /favorites, /settings	Creator/favorite flows	Redirected to /login	Protected without credentials.
/verify-email	Source route exists	Redirected to /login	Deployed behavior differed from file route name.

5.4 Responsive and Interaction Evidence

The source includes desktop navigation and a mobile Sheet navigation menu. Tailwind breakpoint classes such as md:flex, md:hidden, sm:grid-cols-2, and lg:grid-cols-3 define responsive layouts. The team filter has a drawer filter on smaller layouts and an inline filter panel on larger layouts. The hackathon page tracks a selected platform tab and conditionally directs an unauthenticated user to /login when Favorites is selected. These statements are verified by component source. Thirteen genuine user-supplied deployed-interface captures are embedded in Chapter 4, covering public landing, sign-up, profile, navigation, directory, filtering, request, menu, and application-status states. They document rendered states but are not a controlled tablet/mobile viewport test.

5.5 Testing and Validation

The repository includes one Jest suite. It renders a locally defined sample Home component and asserts that a sample heading and paragraph exist. The command npm test -- --runInBand was executed successfully: one test suite passed with two tests. The suite validates the Jest setup but it does not render Synapse app/page.tsx, call a route handler, test authentication, or exercise the deployed service. Accordingly, no broader test success is inferred from it.

ID	MODULE	PROCEDURE	ACTUAL RESULT	STATUS
T01	Unit test setup	Run npm test -- --runInBand.	1 suite; 2 tests passed.	PASS
T02	Public landing route	GET deployed /.	HTTP 200; title Synapse.	PASS
T03	Public team route	GET deployed /teams and /api/teams.	Page 200; API 500.	FAIL
T04	External directories	GET three deployed API handlers.	Unstop, Devpost, Devfolio each 200.	PASS
T05	Protected dashboard	GET /dashboard with redirects.	Final URL /login.	PASS
T06	Favorites API	GET without session.	HTTP 401 Unauthorized.	PASS
T07	Registration validation	Browser form invalid-input interaction.	Browser runtime unavailable.	BLOCKED
T08	UI state evidence	Review supplied deployed captures.	13 genuine captures embedded in Chapter 4.	PASS
T09	Apply workflow	Authenticated submission.	No credentials supplied; no production records created.	BLOCKED

5.6 Results and Discussion

The inspected code and HTTP responses demonstrate an operational public shell, authentication routing, external hackathon aggregation, and a

database-backed model for profiles, team requests, and applications. The three external directory services were live during inspection. The project also provides a clear data structure for role, experience, and skills. These are practical prerequisites for future intelligent decision support.

The strongest FOAI conclusion is that Synapse currently implements a rule-based filtering interface, not an automated constraint solver. A selected search, mode, experience, and skill condition is applied as an AND combination. This is useful search refinement, but it does not select a team from candidates, maximize an objective, or produce recommendations. The present system should be described as constraint-ready rather than CSP-based until a solver or explicit scoring method is implemented and tested.

5.7 Observed Limitations

LIMITATION	EVIDENCE	IMPACT
Team list service failure	Deployed GET /api/teams returned HTTP 500.	Team cards cannot be verified in the live environment.
No verified match algorithm	No Match API or solver; filtering resides in PostsTab.tsx.	AI claim must be limited to rule-based discovery.
Schema/UI naming drift	Schema uses requiredRole/requiredExperience while components use role/experience.	Filter and detail display may not align with current data model.
API authorization concerns	/api/user POST and /api/apply accept user identifiers from client input.	Server-side ownership validation should be added.
Automation limitation	Browser-control runtime could not initialize.	No controlled interactive or multi-viewport test.
Limited tests	Jest suite uses a sample component.	No direct integration or end-to-end coverage.

5.8 Traceability from Claim to Evidence

The report uses three kinds of evidence. Source evidence establishes what the repository contains at the inspected revision. Deployment evidence establishes only what a non-authenticated HTTP request observed at the stated URL. Test evidence establishes only the command and assertions actually

executed. This separation prevents a feature described in a README, architecture diagram, or planned schema comment from being reported as a completed and demonstrated capability.

For example, the README calls the system AI-driven and describes Constraint Satisfaction matching. The schema comments also use CSP language. The executable client filter is the stronger implementation evidence because it supplies the actual predicates and rendering outcome. The report therefore treats role and experience enumerations as constraint-ready fields, while classifying the running filter as a deterministic Boolean narrowing procedure. A future implementation should add a separate matcher module, unit tests with known candidate sets, and recorded decisions that can be checked against the stated constraints.

REPORT CLAIM	PRIMARY EVIDENCE	CONFIDENCE BOUNDARY
Uses PostgreSQL through Prisma	prisma/schema.prisma and lib/prisma.ts	Source-confirmed; provider not inferred from the old architecture image.
Public pages are reachable	HTTP 200 and HTML title observations	Only unauthenticated route reachability was checked.
External directories aggregate data	Three route-handler sources and three HTTP 200 responses	Provider payload freshness was not evaluated.
Team matching is not a CSP solver	Absence of match route/solver; PostsTab predicate implementation	Conclusion is limited to inspected source.
Protected pages require login	middleware.ts plus redirect observation	Authenticated behavior remains untested.

5.9 User Interface Evidence Boundary

The report embeds thirteen genuine user-supplied captures of the deployed Synapse interface in Section 4.8. The capture manifest records their filenames, represented URL or state, viewport dimensions, and CAPTURED status. The report does not label these captures as end-to-end test results: no controlled browser session, form submission, tablet viewport run, or mobile viewport run

was executed in this environment. The browser-control runtime was unavailable during the original capture attempt. This distinction preserves the difference between a real user-interface image, a source-derived diagram, and an automated interaction test.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

Synapse provides a coherent web application structure for hackathon collaboration: public discovery pages, user authentication, a PostgreSQL-backed data model, team-request creation, applications, profile editing, and external hackathon aggregation. The implementation uses current Next.js App Router patterns, Prisma data access, NextAuth sessions, and a responsive component library. The deployed Vercel service returned public pages and live external directory results during inspection, showing that the external event discovery portion was reachable.

For FOAI, the report distinguishes the system's stored constraints from a completed intelligent matcher. Role and experience enums and skill arrays give the system a useful representation for a future CSP. The implemented discovery function currently filters listings with Boolean conditions. It does not solve a CSP, score alternatives, rank matches, or apply machine learning. Presenting that limitation directly protects the technical credibility of the project and gives an achievable next implementation target.

6.2 Recommended Future Work

- Implement an explicit candidate-to-team matcher with declared variables, domains, hard constraints, soft preferences, and an explainable selection result.
- Normalize profile and team requirement field names so client components use requiredRole and requiredExperience consistently.
- Move filtering and pagination to the server when data volume grows, then expose stable query parameters and test the API contract.
- Add session ownership checks to profile and application mutations; validate post bodies with the imported Zod schema.
- Repair and monitor the deployed /api/teams service, including a useful 200 success response and graceful empty-state behavior.

- Add integration tests for route handlers and end-to-end tests for authentication gates, invalid forms, filtering, favorite toggling, and responsive navigation.
- Use a dedicated non-production test account for safe screenshot capture and protected workflow verification.

6.3 Future FOAI Matching Design

A next version can treat each open team position as a variable. Its candidate domain should include active profiles that meet basic eligibility criteria. Hard constraints can enforce the required role, skill coverage, availability, and one-application-per-post condition. Soft constraints can award extra relevant skills, compatible timezones, past collaboration preference, and balance across roles. A deterministic scoring formula should be published with every suggestion, while a CSP or branch-and-bound search can generate feasible combinations. The interface should explain the result in plain language, for example by identifying the fulfilled role, overlapping skills, and unmet preferences.

6.4 Proposed Evaluation Plan

A future matcher should be evaluated with a controlled, non-production dataset rather than a claim of accuracy without ground truth. The dataset can contain profiles with declared roles, skills, experience, timezone, and availability, together with team requirements. Test cases should include an exact skill match, a role match with missing skills, multiple feasible candidates, an empty candidate domain, and conflicting hard constraints. Each case should record the input, generated feasible set, score or objective value where applicable, and the final explanation presented to the user.

The first implementation can remain explainable and deterministic. It can filter candidates by hard constraints, compute skill coverage as the count of required skills that overlap a candidate profile, apply documented bonuses for optional preferences, and sort by score with a stable secondary key. A subsequent team-level solver can select multiple users jointly and reject a

combination when required roles are duplicated or a required role is absent. This staged approach makes the transition from a discovery filter to an FOAI matching module testable, reviewable, and appropriate for student project evidence.

6.5 Acceptance Criteria for the Next Release

The next release should not claim intelligent matching until it satisfies a small, verifiable acceptance set. First, a request must identify its required role, experience level, and skills using the same field names as the database schema. Second, a profile candidate must be evaluated only after the server confirms that the candidate is eligible to view or apply to that request. Third, every generated recommendation must expose the hard constraints that passed, the optional preferences that influenced order, and the reason any candidate was excluded. Fourth, the deployed team endpoint must return either a documented empty collection or a successful collection response; an internal error must be observable through monitoring and test coverage.

A suitable test fixture can use five profiles and two team requirements. It should prove that a missing hard skill excludes a profile, a complete skill overlap ranks above a partial optional overlap, a duplicate application is rejected, and an empty domain produces a clear result. The test suite should run against a disposable PostgreSQL database and should assert the response body as well as status codes. A browser test can then verify that the explanation is visible to the participant without exposing another user’s private email, resume, or authentication token. These criteria connect the FOAI formulation to implementable software behavior.

6.6 Implementation Milestones

The system can be improved in small, evidence-driven milestones. The first milestone is data-contract repair: use requiredRole and requiredExperience consistently in the Prisma model, create form, detail page,

and filter logic. The second is service reliability: identify the root cause of the deployed team-list HTTP 500 response, add structured error logging, and cover the success, empty, and error cases with a route-handler test. The third is authorization hardening: derive mutable user identifiers from the current session rather than accepting them from the client. Each milestone reduces a concrete failure mode before the project adds a more complex matching algorithm.

The next milestone can introduce a pure matching function with no network dependency. Its inputs should be a normalized team requirement and a collection of candidate profiles; its output should be a stable array of result objects containing feasible status, matched skills, unmet hard constraints, optional-preference score, and explanation text. Unit tests can specify a small candidate fixture and assert exact result order. Only after this function is stable should it be exposed through a protected API route and added to the user interface. This sequence keeps the FOAI claim connected to executable, inspectable behavior.

MILESTONE	DELIVERABLE	EVIDENCE OF COMPLETION
Data contract repair	Shared role and experience field types	Form submission, API response, and filter tests use one name.
Team service reliability	GET /api/teams success and error handling	Deployment returns 200 or a documented empty array.
Authorization hardening	Session-derived ownership checks	Negative tests reject mismatched user identifiers.
Rule matcher	Pure deterministic matching function	Fixture tests show feasible set and stable ordering.
CSP extension	Constraint model and explanation UI	Integration tests demonstrate hard and soft constraints.

6.7 Documentation and Deployment Checklist

The final stage of the project should verify that the deployed environment, source repository, and submitted documentation agree. The deployment should expose the same public routes that are described in the report, and every route handler should use the same schema fields named in the client forms. Public information may be indexed for discovery, but protected data such as resumes, account tokens, and private contact information must remain behind an authenticated session. A server error must be treated as a release issue and not as an empty result. This checklist gives the project a repeatable handoff process for future versions.

Before deployment, the team should run the unit suite, add route-handler tests for the team and application endpoints, and use a non-production account to complete safe browser tests. After deployment, the team should confirm the response code for the landing page, team directory, external hackathon directory, and protected dashboard redirect. The report should then be updated only with results that were actually observed. Keeping source evidence, deployment evidence, and user-interface evidence separate avoids overstating the work completed by the current version.

CHECK	EXPECTED EVIDENCE	CURRENT STATUS
Schema consistency	One shared role and experience naming contract	Needs implementation alignment.
Public routes	200 responses for intended public pages	Landing and external APIs observed.
Protected routes	Redirect or authorization result without data leak	Redirect behavior observed.
Team discovery	Successful list response or explicit empty state	Deployment returned HTTP 500.
UI evidence	Genuine deployed-interface captures with captions	13 captures embedded; no controlled multi-viewport test.

CHAPTER 7

REFERENCES

[1] Auth.js, "NextAuth.js documentation," 2026. [Online]. Available: https://authjs.dev. Accessed: Oct. 2026.

[2] R. Barták, "Constraint satisfaction problem," in Encyclopedia of Artificial Intelligence, 2nd ed., Wiley, 2009.

[3] R. Dechter, Constraint Processing. San Francisco, CA: Morgan Kaufmann, 2003.

[4] T. F. Gonzalez, Ed., Handbook of Approximation Algorithms and Metaheuristics. Boca Raton, FL: CRC Press, 2007.

[5] T. Lappas, K. Liu, and E. Terzi, "Finding a team of experts in social networks," in Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009, pp. 467-476.

[6] Next.js, "Next.js 14 App Router documentation," 2024. [Online]. Available: https://nextjs.org/docs/14/app. Accessed: Oct. 2026.

[7] PostgreSQL Global Development Group, "PostgreSQL documentation," 2026. [Online]. Available: https://www.postgresql.org/docs/. Accessed: Oct. 2026.

[8] Prisma, "PostgreSQL database connector," 2026. [Online]. Available: https://www.prisma.io/docs. Accessed: Oct. 2026.

[9] Raghul2808, "Synapse source repository," GitHub. [Online]. Available: https://github.com/Raghul2808/Synapse. Accessed: Oct. 2026.

[10] Synapse, "Deployed application," Vercel. [Online]. Available: https://synapse-livid-gamma-14.vercel.app/. Accessed: Oct. 2026.

[11] Zod, "TypeScript-first schema validation," 2026. [Online]. Available: https://zod.dev. Accessed: Oct. 2026.